

Střední soukromá odborná škola 1.KŠPA v Litoměřicích
obor Informační technologie-IT zaměřený na Programování a Tvorbu webových stránek



Šachy aplikace

Dokumentace projektu

Nguyen Xuan Tiep

IT 4.BI
Leden–Březen 2025

Zadání:

Cílem projektu je vytvořit šachovou aplikaci v jazyce C#, která umožní hraní šachu proti druhému hráči na stejném počítači. Každý hráč bude mít určený čas, který si vyberou hráči. Aplikace bude dodržovat standardní šachová pravidla, včetně speciálních tahů jako je rošáda, en passant a povýšení pěšce. Při opakování tahů nebo 50 tahů bez zachycení šachové figurky určí remízu. Uživatelské rozhraní bude jednoduché a přehledné šachovnice v levé části s polem pro historii pohybů, tlačítka pro uložení hry a časomíry. Další bude funkce pro zpětný tah, který jak název napovídá vrátí hru o jeden tah zpět. Pohyb figurek bude pomocí myši, prvním stiskem vyberete figurku a druhým ji položíte na další pozici. Při vybraní šachové figurky budou zvýrazněné platné tahy. Konec hry bude při šach matu nebo opakování 3 stejných tahů za sebou.

Dílčí úkoly:

- Vytvoření šachovnice
- Vytvoření figurek
- Implementace tahů jednotlivých figurek
- Nastavení pravidel
- Detekce legálních tahů
- Řešení speciálních tahů
- Detekce šachu a šach-matu
- Vytvoření tlačítka pro uložení hry a zpětný tah
- Vytvoření časomíry
- Pole pro historie zobrazení tahů
- Vytvoření startovního okna a okno k ohlášení konce hry

Čestné prohlášení

Prohlašuji, že jsem maturitní práci na téma „ŠACHY V JAZYCE C#“ zpracoval sám. Veškeré prameny a zdroje informací, které jsem použil k sepsání této práce, byly citovány v poznámkách pod čarou a jsou uvedeny v seznamu použitých pramenů a literatury.

Dne

Podpis studenta.....

Obsah

Zadání:	1
Dílčí úkoly:	1
Čestné prohlášení	2
Úvod	5
Popis a v čem pracuji	6
Programovací jazyk	6
Visual Studio	6
PROCREATE	6
Cílová skupina aplikace	6
Jak se hrají šachy	7
Jak jsem pracoval diagramech	8
Use case a aktivity diagram	8
Use case diagram šachy	8
Aktivity diagram Chess bot	9
Aktivity diagram šachy	10
Postup práce	11
Vytvoření hracího pole	11
Vytvoření figurek	11
Uložení figurek do programu	12
Jednoduchá funkce pro pohyb figurek	12
Výpočet tahů	13
Zakomponování vypočítaných tahů do pohybu	13
Detekce figurek v cestě	14
Kreslení figurek na šachovnici	15
Sebrání figurek	15
Vizualizace tahů	15
Třída pro vykonání tahů	16
Speciální tahy	16
Zachycení pěšákem	16
Rošáda	17
En passant	19
Povýšení	20
Vymazání tahů, co nechají svého krále v šachu	21
Detekce konce hry	22
Výpis tahů do listu	24
Výpis sebraných figurek	26

Metoda nová hra	26
Počítačový oponent.....	27
První verze	27
Druhá verze	28
Třetí verze.....	30
Předefinované tahy – čtvrtá verze	35
Uložení/načtení her.....	36
Závěr.....	39
Citace.....	40

Úvod

V dnešní době je téměř každý seznámen s hrou šachy, nebo o ní alespoň slyšel. K jejich popularizaci přispěl trend z roku 2020, kdy se šachy staly populárními mezi širší veřejností na internetu. Téma projektu jsem si vybral, protože jsem byl jedním z lidí, kteří v té době začali hrát šachy, a zajímalo mě, jak se šachy programují. Další věcí, která mě zajímala, bylo, jak fungují šachové stroje (Chess Engine) a jak mohou nahlížet několik stovek tahů dopředu během krátké doby.

Cílem projektu bylo, jak jsem již zmínil, zjistit, jak fungují šachové stroje, a zároveň pochopit strukturu objektově orientovaného programování ve větším projektu. Samozřejmě mě zajímala i logika hodnocení tahů a detekce šachmatu.

K práci na projektu jsem použil Visual Studio a C# jako součást .NET Framework. Další aplikací, kterou jsem využil, byl Procreate.

Tato dokumentace popisuje, na čem jsem pracoval, a jak jsem postupoval při vývoji aplikace. Je rozdělena na teoretickou a praktickou část, přičemž každá z nich dále rozděluje jednotlivé části projektu.

Hlavní funkcí aplikace je možnost hrát šachy proti sobě nebo proti počítači. Hru lze uložit a později znovu načíst, pokud nelze dále pokračovat. Po šachmatu nebo remíze se hra resetuje.

Teoretická část

Popis a v čem pracuji

Programovací jazyk

Pro svůj projekt jsem si vybral jazyk C#, protože to byl jediný jazyk, který jsem ovládal kromě JavaScriptu, a ten bych zde stejně nepoužil.^[1]

C# je vysokoúrovňový programovací jazyk od firmy Microsoft. Vysokoúrovňový programovací jazyk je řazení programovacího jazyka podle toho, jak moc se podobá lidské řeči a tím je jednodušší s ním pracovat další jazyk ve stejné kategorii je třeba Python.

Visual Studio

Pro svoje vývojové prostředí jsem si vybral Visual Studio, protože jsem s ním měl nejvíce zkušeností a je jednodušší s ním pracovat v C#. ^[2]

Microsoft Visual Studio je vývojové prostředí (IDE) od Microsoftu. Může být použito pro vývoj konzolových aplikací a aplikací s grafickým rozhraním.

Visual Studio obsahuje editor kódu podporující IntelliSense a refaktorování. Integrovaný debugger pro tvorbu aplikací s GUI.

Visual Studio podporuje jazyky prostřednictvím jazykových služeb, což umožňuje, aby editor kódu a debugger podporoval jakýkoliv programovací jazyk. Mezi vestavěné jazyky patří C/C++/C#.

PROCREATE

Tato aplikace je rastrový grafický editor nebo jinak aplikace pro digitální kreslení a animaci. Je to vlastně jako malování ve Windows akorát s více nástroji. V této aplikaci jsem vytvořil obrázky figurek pro mojí aplikaci.^[3]

Cílová skupina aplikace

Jak vyplývá z názvu, aplikace je cílená na lidi, co by si rádi zahráli šachy ve svém volném času proti kamarádovi nebo jednoduchému počítači.

Jak se hrají šachy

Šachy jsou strategická desková hra pro dva hráče, která se hraje na šachovnici o rozměru 8×8 polí. Každý hráč začíná s 16 figurami: jedním králem, jednou dámou, dvěma věžemi, dvěma střelci, dvěma jezdci a osmi pěšci. Hráči se střídají v tazích, přičemž každá figura má přesně daný způsob pohybu. Cílem hry je dát soupeřovu králi mat – tedy ho napadnout tak, aby se nemohl bránit žádným legálním tahem. Hra může skončit i remízou, například pokud dojde k patové situaci nebo se třikrát zopakuje stejná pozice na šachovnici.

Pěšci se pohybují pouze dopředu o jedno pole, v prvním tahu mohou o dvě, a berou diagonálně. Jezdci se pohybují do tvaru „L“ a jsou jediní, kteří mohou přeskakovat jiné figury. Střelci se pohybují diagonálně libovolně daleko, věže po slupcích a řadách, dáma může kombinovat pohyb věže i střelce a král se posouvá o jedno pole libovolným směrem. Existují i speciální tahy – rošáda, při které se král a věž v jednom tahu vymění, braní mimochodem při pohybu pěšců a proměna pěšce, když dojde na poslední řadu soupeře.

Šachy jsou hrou logiky a strategie, kde je klíčové předvídat soupeřovy tahy a plánovat několik tahů dopředu. Mistrovské zvládnutí šachu vyžaduje nejen talent, ale také pravidelný trénink a zkušenosti.

Praktická část

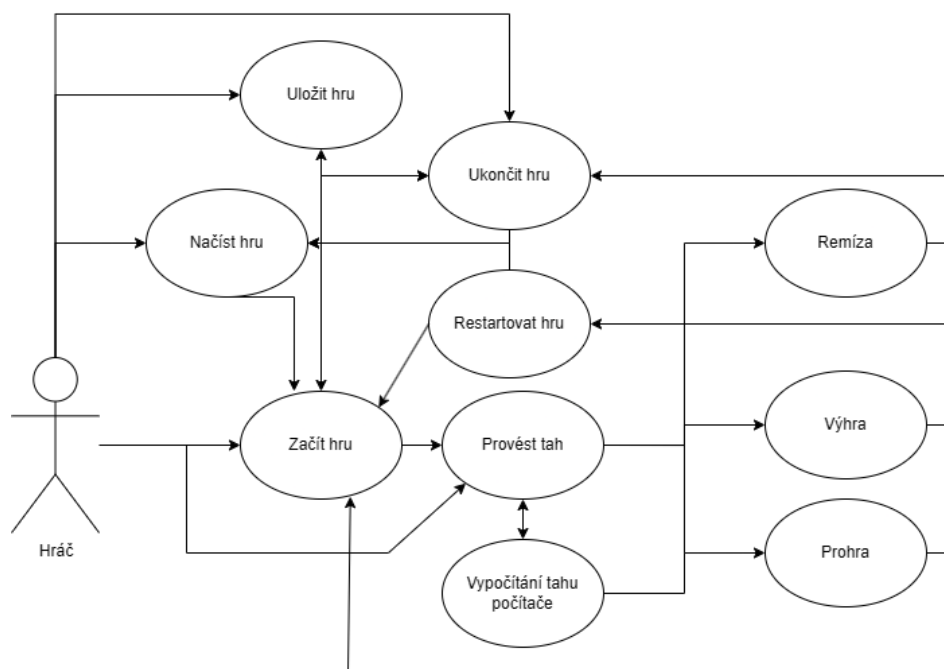
Jak jsem pracoval diagramech

Na diagramech jsem pracoval v aplikaci drawio. Kostru diagramů jsem vzal z této dokumentace a projektu. [\[13\]\[14\]\[15\]](#)

Use case a aktivity diagram

Use case diagram šachy

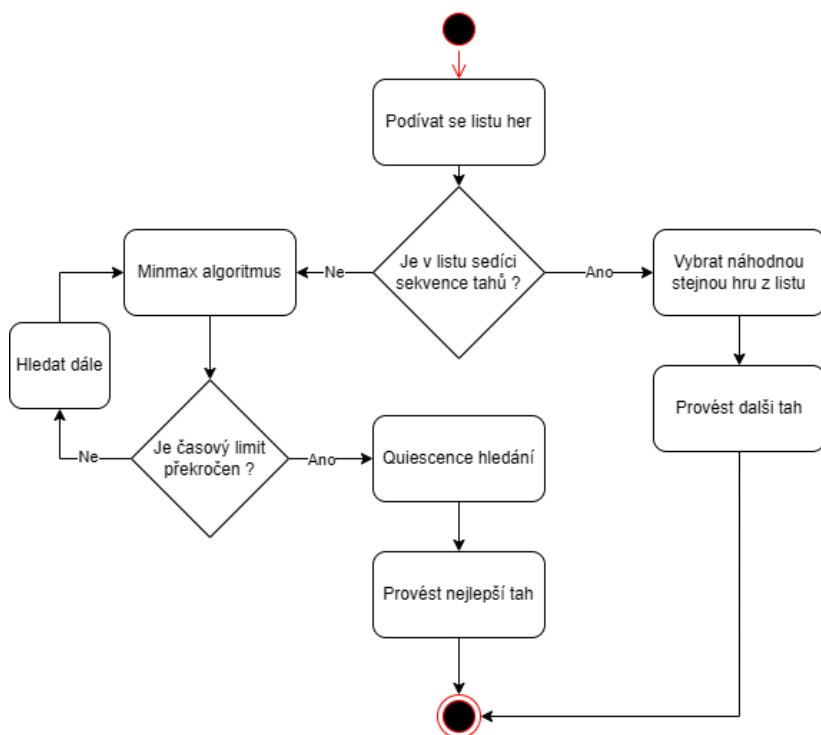
Tento diagram znázorňuje základní funkce šachové aplikace. Uživatel má možnost uložit aktuální stav hry, aby ji mohl později pokračovat, nebo načíst dříve uloženou partii. Pro začátek nové hry slouží funkce začít hru, která inicializuje šachovnici a figury do výchozího stavu. Pokud hráč potřebuje ukončit aktuální partii, může vypnutím aplikace, nebo ji restartovat pro rychlý začátek nového zápasu. Během hry může hráč provádět tahy manuálně, nebo může nechat vypočítat tah počítače, pokud hraje proti AI. Aplikace také umí řešit koncové stavy hry. V případě, že partie skončí nerozhodně, například patem nebo opakováním pozic, označí se jako remíza. Pokud jeden z hráčů dosáhne vítězné pozice, aplikace určí výhru nebo prohru podle toho, kdo byl matován nebo se vzdal.



Obrázek 1 - Use case diagram šachy

Aktivity diagram Chess bot

Tento diagram znázorňuje proces rozhodování šachového bota (AI) během hry. Bot nejprve zkontroluje seznam předdefinovaných her, aby zjistil, zda existuje sekvence tahů odpovídající aktuální situaci na šachovnici. Pokud ano, vybere náhodně jednu z těchto her a provede další tah podle této sekvence. Pokud ne, přepne se na použití Minmax algoritmu, který simuluje možné tahy a protitahy do určité hloubky, aby našel optimální tah. Během tohoto procesu bot kontroluje, zda byl překročen časový limit pro výpočet tahu. Pokud ano, provede Quiescence search, což je minmax, ale prochází všechny tahy, kde může sebrat figuru. Poté provede nejlepší dosud nalezený tah. [\[7\]\[10\]](#)

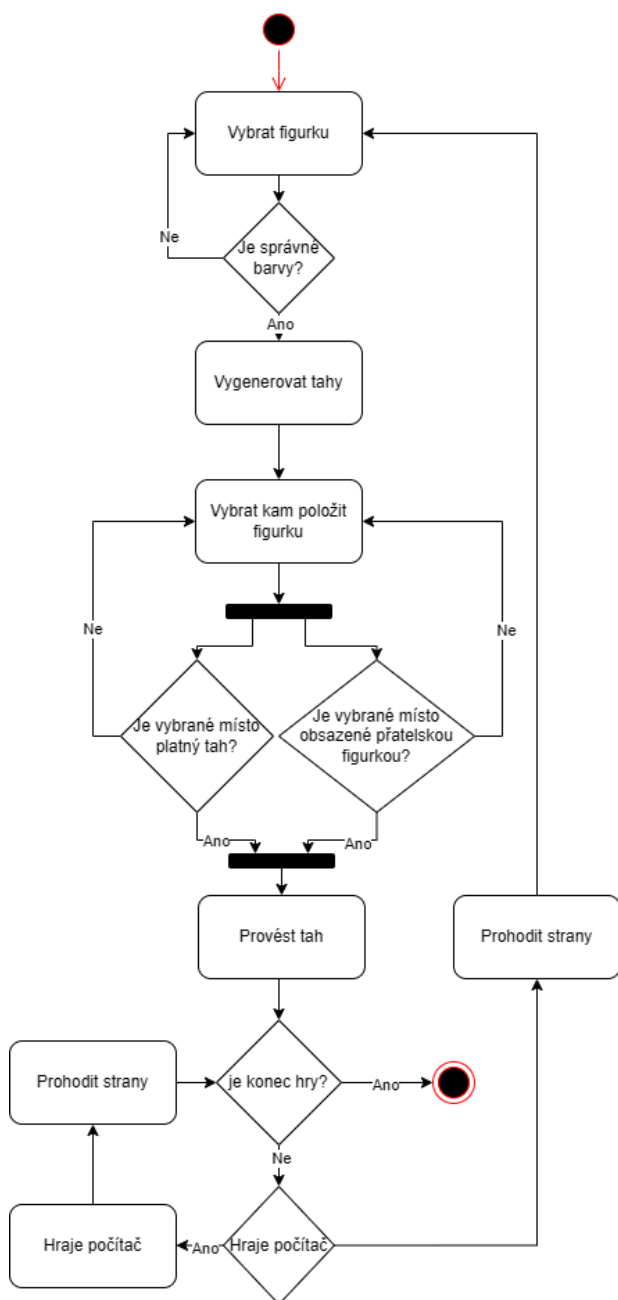


Obrázek 2 - Aktivita diagram Chess bota

Aktivity diagram šachy

Tento diagram znázorňuje proces hraní šachové partie, a to jak pro lidského hráče, tak pro počítačového oponenta. Hráč nejprve vybere figurku, se kterou chce provést tah, a systém zkontroluje, zda je figurka správné barvy. Pokud ano, vygeneruje všechny platné tahy pro tuto figurku. Hráč pak vybere cílové pole, kam chce figurku přesunout, a systém zkontroluje, zda je tento tah platný a zda na cílovém poli není přátelská figurka. Pokud jsou všechny podmínky splněny, figurka je přesunuta.

Poté systém zkontroluje, zda hra skončila (šachmat, remíza atd.). Pokud ne, hra pokračuje a tah je předán druhému hráči (lidskému nebo počítačovému). Pokud hraje počítač, provede svůj tah pomocí předdefinovaných algoritmů, jako je Minmax, a poté opět přepne strany. Tento cyklus se opakuje, dokud hra nedospěje ke konci.



Obrázek 3 - aktivity diagram šach

Postup práce

Vytvoření hracího pole

Začal jsem vytvořením projektu ve Visual studiu a použil jsem .NET framework template. Prvně jsme si myslel, že si nakreslím hrací pole a pozice figurek bude podle mřížky, kterou bych dal na hrací pole. Nedokázal jsem to však tuto myšlenku přenést do funkčního kódu. Nakonec jsem se rozhodl udělat hrací pole z tlačítek a kvůli své lenosti jsem jejich vytvoření udělat v kódu. To mi pomohlo potom s rozložením aplikace, když bych potřeboval změnit velikosti políček v hracím poli. Pro lepší vzhled políček jsem dal tlačítka na placatého vzhled. V kódu je to implementováno dvěma FOR cykly, jeden pro sloupce a druhý pro řádky. Pozice tlačítek se vypočítává z nuly a pak se přičte velikost tlačítka krát kolikáté tlačítko tvoříme. Barva se potom rozhoduje podle pozice tlačítka XY, což se přičte a podle výsledku se vybere barva pro sudé nebo liché číslo. Potom se přidá do tlačítka klik metoda pro pozdější hýbání s figurkami. Nakonec se přidá do formu a do listu tlačítek pro její pozdější úpravu. List jsme později musel změnit na 2D pole pro generování tahů.

```
for (int y = 0; y < poleSize; y++)
{
    for (int x = 0; x < poleSize; x++)
    {
        Button button = new Button
        {
            Size = new Size(velikostPolicka, velikostPolicka),
            Text = null,
            BackgroundImageLayout = ImageLayout.Zoom
        };
        if (x == 0)
        {
            button.Location = new Point(15, y * velikostPolicka + 30 + cernySebran.Height);
        }
        else
        {
            button.Location = new Point(x * velikostPolicka + 15, y * velikostPolicka + 30 + cernySebran.Height);
        }
        button.FlatStyle = FlatStyle.Flat;
        button.FlatAppearance.BorderSize = 0;
        if ((y + x) % 2 == 0)
        {
            button.BackColor = barvaSuda;
            button.FlatAppearance.MouseOverBackColor = barvaSuda;
            button.FlatAppearance.MouseDownBackColor = barvaSuda;
        }
        else
        {
            button.BackColor = barvaLicha;
            button.FlatAppearance.MouseOverBackColor = barvaLicha;
            button.FlatAppearance.MouseDownBackColor = barvaLicha;
        }
        button.Click += pole_Click;
        this.Controls.Add(button);
        hraciPole[y, x] = button;
    }
}
```

Obrázek 4 - kód generace šachovnice

Vytvoření figurek

Figurky jsou udělané jako PNG. Inspiraci jsem si vzal z klasických figurek, co jsou na stránce Chess.com. Figurky jsou více kostkaté a jejich "hlavy" levitují nad jejich těly. Použil jsem k tomu program s názvem Procreate, což je rastrový grafický editor pro digitální malbu. Raději jsem požil tento program než malování ve Windows, nabízelo mi to více nástrojů ke tvorbě. [\[4\]\[3\]](#)



Pěšák

Uložení figurek do programu

Figurky jsem potom uložil jako materiál v programu, aby nebral figurky podle cesty k souboru. Vytvořil jsem si pak novou třídu pojmenovanou třída figurky. V té jsem si udělal slovník pro přístup k figurkám. Každá figurka pak má klíč podle kódu figurky. Jako poslední jsem tam přidal funkci, co vrátí obrázek ze slovníku podle kódu figurky.

```
Dictionary<string, Image> CestakFigurkam = new Dictionary<string, Image>()
{
    {"k", šachys.Properties.Resources.KingB},
    {"K", šachys.Properties.Resources.KingW},
    {"q", šachys.Properties.Resources.QueenB},
    {"Q", šachys.Properties.Resources.QueenW},
    {"b", šachys.Properties.Resources.BishopB},
    {"B", šachys.Properties.Resources.BishopW},
    {"r", šachys.Properties.Resources.RookB},
    {"R", šachys.Properties.Resources.RookW},
    {"p", šachys.Properties.Resources.PawnB},
    {"P", šachys.Properties.Resources.PawnW},
    {"n", šachys.Properties.Resources.KnightB},
    {"N", šachys.Properties.Resources.KnightW},
};
Počet odkazů: 17
public Image rozpoznaniFigurek(string kodFigurky)
{
    return CestakFigurkam[kodFigurky];
}
```

Jednoduchá funkce pro pohyb figurek

Obrázek 5 - slovník všech figurek a jejich vrácení

První jsem si chtěl napsat funkci pro pohyb figurek kdy si vyberete figurku a dalším kliknutím ji položit. Při kliknutí na pole s figurkou se daný kód figurky uloží do proměnné „vybraná figurka“. S tou se pak pracuje při dalším kliknutí prázdného políčka, na které se položí. Položení je v kódu jako úprava dříve zmíněného pole s tlačítky. Souřadnice se zjišťují pomocí Indexof Listu tlačítek. Souřadnice figurky se při vybrání uloží do proměnných, aby bylo možné při položení smazat figurku na správných souřadnicích. Figurka se teda položí na pole, co se kliklo. Figurky lze, ale položit kam chcete na hracím poli. Ke konci jsem přidal jednotlivé metody pro řešení tahů.

Přidání checkru, kdo je na řadě

Po tom, co jsem si vyzkoušel, jestli kód pro pohyb funguje správně jsem přidal IF příkazy. Ty měli na starosti, že černý mohl hýbat jenom s černými figurkami a naopak. Udělal jsem to pomocí bool proměnné, která reprezentovala, kdo byl na řadě. To se potom společně s kódem figurky zkontrolovalo a boolem, jestli to je správná figurka. Černý mohl hýbat jenom s figurkami, co měla kód figurky malým písmem a bílý jenom s figurkami, co měla kód velkým písmem. Nakonec se při položení figurky změní bool naopak.

```
if (bilyNaRade == true && figurkaTag == figurkaTag.ToUpper())
```

Obrázek 6 - podmínka pro bílé figurky

```
else if (bilyNaRade == false && figurkaTag == figurkaTag.ToLower())
```

Obrázek 7 - podmínka pro černé figurky

```
if (bilyNaRade == true)
{
    bilyNaRade = false;
}
else
{
    bilyNaRade = true;
}
```

Obrázek 8 - změni kdo je na řadě

Výpočet tahů

Jako další, jsem začal pracovat na funkci výpočtu tahů figurek. Jako první jsem implementoval jednoduché odčítání nebo sčítání z pozice figurky. Měl jsem totiž list tlačítek a ne pole. Například máme bílého pěšáka a chceme ho posunout o jednu dopředu, v kódu by byl výpočet aktuálního čísla pole $+8$, protože je hrací pole 8×8 . Takhle jsem postupoval stejně u ostatních figurek. Všechny až na pěšáka, protože se hýbou podle strany, na které jsou, jsou tahy stejné na obou stranách ve výpočtu. Bohužel jsem zjistil, že tento způsob přestane

fungovat když, začnu řešit horizontální tahy tak pak vidím tahy jinde, než by měli být. Tak jsem místo listu tlačítek

tak jsem použil dvou rozměrné pole pro lepší manipulaci tahů. To funguje pomocí dvou os, vertikální a horizontální, to se potom odčítá nebo sčítá podle směru tahu. Takže je pěšák na pozici $[1,0]$ a chce se posunout o jeden do předu, tah se vypočítá součtem $[1,0]$ a $[+1,0]$. což vyjde $[2,0]$ nová pozice. Přidal jsem i limit, aby tahy nebyly mimo pole. Funkce potom vrátí list všech tahů konkrétní figurky, která byla vybrána.

Zakomponování vypočítaných tahů do pohybu

Upravil jsem funkci na přemísťování figurek tak aby se dali jen na ty políčka, která jsou správná pro ni. Část kódu pro vybírání figurky je pořád ten samý jenom jsem upravil kód pro její položení. To funguje tak, že se načtou legální tahy z a projdou se všechny tahy v listě a jestli se rovná tah s vybraným políčkem tak se pokračuje jako normálně.

```
Dictionary<string, int[]> legalniTahy = new Dictionary<string, int[]>()
{
    {"p", [8]},
    {"P", [-8]},
    {"k", [-1,1, 8,-8, 9,-9]},
    {"K", [-1,1, 8,-8, 9,-9]},
    {"q", [-8]},
    {"Q", [-8]},
    {"b", [-8]},
    {"B", [8]},
    {"r", [-1,-2,-3,-4,-5,-6,-7, 1,2,3,4,5,6,7]},
    {"R", [-1,-2,-3,-4,-5,-6,-7, 1,2,3,4,5,6,7]},
    {"-", [-8]},
    {"-", [-8]},
};
Počet odkazů: 0
public List<int> vypocetTahu(int cisloAktualnihoPole, string kodFigurky)
{
    List<int> tahy = new List<int>();
    for (int i = 0; i < legalniTahy[kodFigurky].Length; i++) {
        tahy.Add(legalniTahy[kodFigurky][i]);
    }

    for (int i = 0; i < tahy.Count; i++)
    {
        tahy[i] = cisloAktualnihoPole + tahy[i];
    }
    return tahy;
}
```

Obrázek 9 - generace tahů

```

else if (vybranaFigurka != null)
{
    List<int> legalniTahy = TahFigurka.vypocetTahu(souradniceMinulehoPolicka, vybranaFigurka, stavHracihoPole);
    souradniceAktualnihoPolicka = kordFinder(sender as Button);
    for (int x = 0; x < legalniTahy.Count; x+=2)
    {
        if (souradniceAktualnihoPolicka[0] == legalniTahy[x] && souradniceAktualnihoPolicka[1] == legalniTahy[x + 1])
        {

```

Obrázek 10 - upravený kód pro položení

Detekce figurek v cestě

Většina figurek nemohou jít přes ostatní figurky, až na koně, tak jsem upravil generaci tahů, aby bral v potaz, jestli je směr tahu blokován.

Kód prochází každý směr tahů (nahoru, dolů doleva, doprava) a každý tah v tom směru, když narazí na pole s figurkou přidá souřadnice předchozích polí ve směru a přeskočí na další směr tahů. Pokud je pole obsazené nepřátelskou figurkou tak se přidá i to obsazené do listu pro tahy kde ji můžete sebrat. Přidal jsem i pole pro reprezentaci obsazených polí jako 1 a prázdné 0. Přeskakování je přes příklad v „strany Pohybu“

```

int stranyPohybu = ((i / 14) + 1) * 14;
i = stranyPohybu - 2;

```

kde se vytelí aktuální číslo v listu čtrnácti a přičte se jedna, čímž nám vyjde, jaká se teď prochází a přičte se čtrnáct pro přeskočení na další směr, ke konci se ještě odečte dva, aby nám index nevyjel z velikosti listu na konci. Všechny výsledky jsou v celých číslech.

Obrázek 11 - přeskakování na nový směr

```

if (stavFigurek[y, x] != 1)
{
    tahy.Add(y);
    tahy.Add(x);
}
else
{
    if (hraciPole[y, x].Tag != null &&
        ((Char.IsUpper(hraciPole[y, x].Tag.ToString()[0]) && !Char.IsUpper(tempKodFigurky[0])) ||
        (!Char.IsUpper(hraciPole[y, x].Tag.ToString()[0]) && Char.IsUpper(tempKodFigurky[0])))
    {
        tahy.Add(y);
        tahy.Add(x);
    }
}

```

Obrázek 12 - přidání platných tahů do listu tahů

Kreslení figurek na šachovnici

```
hraciPole[7, 4].BackgroundImage = figurka.rozpoznavaniFigurek("K".ToString());  
hraciPole[7, 4].Tag = "K";
```

Obrázek 13 - kreslení figurky krále na pole 7,4

Do této doby jsem jenom vykresloval figurky ručně jak jde vidět na obrázku. Potřeboval jsem jej na testy tahů jednotlivých figurek. Pro vykreslení všech figurek na šachovnici jsem si vytvořil samostatnou funkci, která přímá textový řetězec jako pozice figurek.

Funkce tedy přijme textový řetězec a aktuální šachovnici. Řetězec je ve formátu kód figurky a její pozice seřazeno řádek a sloupec.

```
'r00r07n01n06b02b05q03k04p10p11p12p13p14p15p16p17
```

Obrázek 15 – textový řetězec šachovnice

Na začátku šachovnici vyčistí, aby byla prázdná, totéž udělá i stavu pole. Poté se zpracuje textový řetězec. Textový řetězec prochází pomocí přes FOR cyklus a přeskakuje v inkrementech tří. Ze řetězce se kód figurky podle aktuálního indexu cyklu a na dalších dvou pozicích se vezmou

souřadnice. Souřadnice se musí převést z charakteru do čísel pomocí odečtení ASCII charakteru nula, aby se převedli správně. A na

šachovnici se pomocí kódu figurky a souřadnic vykreslí figurka. Nakonec se na stejné pozici ve stavu pole změní číslo na jedna, aby reprezentovala obsazené políčko.

```
foreach (Button v in tempHraciPole)  
{  
    v.BackgroundImage = null;  
    v.Tag = null;  
}  
  
for (int y = 0; y < stavHracihoPole.GetLength(0); y++)  
{  
    for (int x = 0; x < stavHracihoPole.GetLength(1); x++)  
    {  
        stavHracihoPole[y, x] = 0;  
    }  
}
```

Obrázek 14 - resetování šachovnice a stavu

Sebrání figurek

Jako další jsem přidal funkci, která se zabývá zachycením/sebráním figurek oponenta. V kódu je podmínka, která když

```
for (int i = 0; i < PostaveniFigurek.Length; i += 3)  
{  
    char yT = PostaveniFigurek[i + 1];  
    char xT = PostaveniFigurek[i + 2];  
    int y = yT - '0';  
    int x = xT - '0';  
    char pismeno = PostaveniFigurek[i];  
    tempHraciPole[y, x].BackgroundImage = figurka.rozpoznavaniFigurek(pismeno.ToString());  
    tempHraciPole[y, x].Tag = pismeno.ToString();  
    stavHracihoPole[y, x] = 1;  
}
```

Obrázek 16 - cyklus kreslící figurky na šachovnici

je políčko obsazené zkontroluje, jestli to je figurka oponenta. Z pole se vezme tag a zjistí se, pokud je velkým nebo malým písmem spolu s kým je na řadě. Pokud tyto podmínku pustí tak se položí figurka. Je to zde aby se zabránilo sebrání vlastní figurky. Položení figurek jsem dal do vlastní metody pro přehlednosti. Snímek z kódu jsem si zapomněl uložit, proto ho zde nemám.

Vizualizace tahů

Do aplikace jsem přidal funkci, která při vybrání figurky změní barvy platných tahů. Vygenerují se platné tahy pro danou figurku a přes cyklus se projedou všechny tahy a změní se barva jejich políček na červenou. Po položení nebo zrušení výběru se spustí metoda která, změní barvy políček zpátky.


```
private void mozneTahy(Button vybranaFigurkaButton)
{
    List<int> temp = detekceSachu.tahyProtiSachuMatu(stavHracihoPole, hraciPole,
    for (int i = 0; i < temp.Count; i += 2)
    {
        if (temp[i] < 8 && temp[i] >= 0 && temp[i + 1] < 8 && temp[i + 1] >= 0)
        {
            hraciPole[temp[i], temp[i + 1]].BackColor = Color.Red;
        }
    }
}
```

Obrázek 17 - vizualizace platných tahů

```
private void resetovaniBarevPolicek()
{
    foreach (var v in hraciPole)
    {
        if (v.BackColor == Color.Red || v.BackColor == Color.Blue)
        {
            int[] temp = realizacePohybu.hledacSouradnic(v, hraciPole);
            v.BackColor = ((temp[0] + temp[1]) % 2 == 0) ? barvaSuda : barvaLicha;
        }
    }
}
```

Obrázek 18 - resetování barev šachovnice

Třída pro vykonání tahů

Pro metody, které vykonají tah jsem dal do vlastní třídy pro lépe upravitelný kód. Třída se jmenuje realizace pohybu a obsahuje metodu pro klasické položení figurek a později i metody pro speciální tahy.

Speciální tahy

Zachycení pěšákem

Sebrání pěšákem je speciální, protože nemůže normálně sebrat figurku před ní, pouze figurky na vedlejších předních políčkách. Přidal jsem do generace pohybů tahy pro sebrání. Tahy byly uloženy do slovníku tahů pod klíčem „p3“. samotnou generaci těchto tahů jsem přidal do vlastní metody. Metoda se spustí při generaci tahů pro pěšáky. A přidá tah do listu jenom pokud na této pozici je figurka protivníka. Nakonec se vrátí tento list do hlavní metody a generují se ostatní tahy.

```
if (kodFigurky == "p" || kodFigurky == "P")
{
    tahy = pesakCapture(cisloAktualnihoPole, kodFigurky, hraciPole);
}
```

Obrázek 19 - spuštění metody pro sebrání pěšákem

```

int y = cisloAktualnihoPole[0] + legalniTahy[kodFigurky + "3"][0];
int x = cisloAktualnihoPole[1] + legalniTahy[kodFigurky + "3"][1];
if (x < 8 && x >= 0 && y < 8 && y >= 0)
{
    if (hraciPole[y, x].Tag != null)
    {
        if ((Char.IsUpper(hraciPole[y, x].Tag.ToString()[0]) && !Ch
        {
            tahy.Add(y);
            tahy.Add(x);
        }
    }
}

```

Obrázek 20 - generace, kontrola a přidání tahu

Rošáda

Zjištění, pokud se král nebo věž již neposunula

Rošáda není možná pokud král nebo věž již vykonali tah. Rošádu nelze také pokud jsou v cestě mezi nimi figurky.

Vytvořil jsem si boolean pro černého a bílého krále, pokud se od začátku hry nevykonali pohyb. Další jsem si vytvořil string pole pro zaznamenávání pohybu věží. Při každém pohybu se spustí metoda pro detekci pohybu těchto figurek. Te změní booleany králů na pravda podle kódu krále. Pokud byla figurka věž zjistí se pole, na kterém byly před pohybem a zaznamená se do adekvátního indexu „RookMoves“ pole.

```

if (kodfigurka.ToLower() == "k")
{
    if (!Char.IsUpper(kodfigurka[0]))
    {
        KingBMove = true;
    }
    else
    {
        KingWMove = true;
    }
}

```

Obrázek 22 - detekce, pokud byl král posunut

```

if (!Char.IsUpper(kodfigurka[0]))
{
    if (pole[0] == 0 && pole[1] == 0)
    {
        RookMoves[0] = "N";
    }
    else if (pole[0] == 0 && pole[1] == 7)
    {
        RookMoves[1] = "N";
    }
}
else
{
    if (pole[0] == 7 && pole[1] == 0)
    {
        RookMoves[2] = "N";
    }
    else if (pole[0] == 7 && pole[1] == 7)
    {
        RookMoves[3] = "N";
    }
}

```

Obrázek 21 - detekce, pokud byla věž posunuta

Zjištění, pokud v cestě nepřekáží figurka

Jak jsem zmiňoval ve předešlé části, rošáda není možná pokud královi a věží nepřekáží figurky.

Při pokusu o provedení rošády se zkontroluje, pokud v mezi dvěma figurkami není překážka. V kódu se vyhodnotí, na jakou stranu bude rošáda podle toho, pokud je číslo slupce krále je větší, než sloupec věže v tom případě to je na stranu královny a pokud menší tak strana krále.

Kód projde políčka mezi králem a věží a jestli narazí na obsazené políčko vrátí nepravdu. Projde-li políčka, aniž by narazil na obsazené tak vrátí pravdu,

```
if (pole[1] < pole2[1])
{
    for (int i = pole[1] + 1; i < pole2[1]; i++)
    {
        if (hraciPole[pole[0], i].BackgroundImage != null)
        {
            return false;
        }
    }
}
```

Obrázek 24 - kontrola překážek (strana krále)

```
else
{
    for (int i = pole[1] - 1; i > pole2[1]; i--)
    {
        if (hraciPole[pole[0], i].BackgroundImage != null)
        {
            return false;
        }
    }
}
return true;
```

Obrázek 23 - kontrola překážek (strana královny)

Vykonání rošády

Pokud se splní všechny podmínky pro rošádu projde ještě poslední podmínkou ta zkontroluje, pokud jsou figurky na správných místech. Poté se vykoná rošáda.

```
if (!KingBMove && pole2[0] == 0 && pole2[1] == 0 && RookMoves[0] == null)
{
    hraciPole = castelingMove(pole, pole2, vybranaFigurka, aktualniFigurka, hraciPole);
}
else if (!KingBMove && pole2[0] == 0 && pole2[1] == 7 && RookMoves[1] == null)
{
    hraciPole = castelingMove(pole, pole2, vybranaFigurka, aktualniFigurka, hraciPole);
}
else if (!KingWMove && pole2[0] == 7 && pole2[1] == 0 && RookMoves[2] == null)
{
    hraciPole = castelingMove(pole, pole2, vybranaFigurka, aktualniFigurka, hraciPole);
}
else if (!KingWMove && pole2[0] == 7 && pole2[1] == 7 && RookMoves[3] == null)
{
    hraciPole = castelingMove(pole, pole2, vybranaFigurka, aktualniFigurka, hraciPole);
}
```

Obrázek 25- kontrola správných souřadnic figurek

```
if (pole2[1] > pole[1])
{
    hraciPole[pole[0], pole[1] + 1].BackgroundImage = TridaFigurky.rozpoznavaniFigurek(aktualniFigurka);
    hraciPole[pole[0], pole[1] + 1].Tag = aktualniFigurka;
    hraciPole[pole2[0], pole2[1] - 1].BackgroundImage = TridaFigurky.rozpoznavaniFigurek(vybranaFigurka);
    hraciPole[pole2[0], pole2[1] - 1].Tag = vybranaFigurka;
    hraciPole[pole[0], pole[1]].BackgroundImage = null;
    hraciPole[pole[0], pole[1]].Tag = null;
    hraciPole[pole2[0], pole2[1]].BackgroundImage = null;
    hraciPole[pole2[0], pole2[1]].Tag = null;
}
else
{
    hraciPole[pole[0], pole[1] - 1].BackgroundImage = TridaFigurky.rozpoznavaniFigurek(aktualniFigurka);
    hraciPole[pole[0], pole[1] - 1].Tag = aktualniFigurka;
    hraciPole[pole2[0], pole2[1] + 2].BackgroundImage = TridaFigurky.rozpoznavaniFigurek(vybranaFigurka);
    hraciPole[pole2[0], pole2[1] + 2].Tag = vybranaFigurka;
    hraciPole[pole[0], pole[1]].BackgroundImage = null;
    hraciPole[pole[0], pole[1]].Tag = null;
    hraciPole[pole2[0], pole2[1]].BackgroundImage = null;
    hraciPole[pole2[0], pole2[1]].Tag = null;
}
```

Obrázek 26 - vykonání rošády

En pasant

en pasant je tah kdy pěšák může sebrat pěšáka oponenta, když se posune za něj. Nepřátelský pěšák se však musel minulý tah posunout o dvě pole. Pěšáka lze vzít pouze svým pěšákem a musí být hned na vedlejším poli. en pasant nelze vykonat, pokud byl předminulý tah (Černý posune pěšáka o 2, ale na dalším tahu ho bílý nesebere nebo nemůže, potom en pasant stane nevalidním).

Pro en pasant jsem si vytvořil pole pro uložení souřadnic daného pěšáka. Při tahu, kde se pěšák posune o dvě pole se aktuální souřadnice figurky uloží do en pasant pole. Na konci se poté odečte/přičte jedna, aby byla souřadnice za pěšákem.

```
if (figurka.ToLower() == "p" && Math.Abs(aktualni[0] - minuly[0]) == 2)
{
    Array.Clear(EnPeasant, 0, 2);
    EnPeasant[0] = aktualni[0];
    EnPeasant[1] = aktualni[1];
    if (Char.IsUpper(vybranaFigurka[0]))
    {
        EnPeasant[0] += 1;
    }
    else
    {
        EnPeasant[0] -= 1;
    }
}
```

Obrázek 27 - detekce posunutí pěšáka o dvě pole

```
bool SpravnaFigurkaNaEnPeasant()
{
    return (souradniceMinulehoPolicka[0] == EnPeasant[0] - 1 && (souradniceMinulehoPolicka[1] == EnPeasant[1] - 1 ||
        (souradniceMinulehoPolicka[0] == EnPeasant[0] + 1 && (souradniceMinulehoPolicka[1] == EnPeasant[1] - 1 ||
```

Obrázek 28 - kontrola správného pěšáka pro en pasant (1/2)

```
souradniceMinulehoPolicka[1] == EnPeasant[1] + 1) && vybranaFigurka == "p") ||
souradniceMinulehoPolicka[1] == EnPeasant[1] + 1) && vybranaFigurka == "p");
```

Obrázek 29 - kontrola správného pěšáka pro en pasant (2/2)

Pokud se hned na dalším tahu vybere pěšák, zkontroluje se, pokud je správná figurka pro vykonání en passant. Pokud je pěšák správná figurka tak se zobrazí en passant tah. Při položení pěšáka na en passant se znovu zkontroluje, pokud je en passant pole větší než 0 (en passant na poli 0 nejde, tak se používá jako null). Poté se zkontroluje, pokud je figurka správná na en passant a položí se pomocí metody speciální en passant.

```
if (EnPeasant[0] > 0 && spravnaFigurkaNaEnPeasant())
{
    if (detekceSachu.testTahuProtiSachuMatuEnPeasant(stavHracihoPole,
    {
        hraciPole[EnPeasant[0], EnPeasant[1]].BackColor = Color.Red;
    }
}
```

Obrázek 30 - vybarvení políčka en passant

```
if (souradniceAktualnihoPolicka[0] == EnPeasant[0] && souradniceAktualnihoPolicka[1] == EnPeasant[1])
{
    if (spravnaFigurkaNaEnPeasant() && detekceSachu.testTahuProtiSachuMatuEnPeasant(stavHracihoPole, h
    {
        if (Char.IsUpper(vybranaFigurka[0]))
        {
            EnPeasant[0] += 1;
        }
        else
        {
            EnPeasant[0] -= 1;
        }
        hraciPole = realizacePohybu.enPeasantCapture(vybranaFigurka, hraciPole[EnPeasant[0], EnPeasant
        enPeasantChecker = true;
        Array.Clear(EnPeasant, 0, 2);
    }
}
```

Obrázek 31 - Kontrola en passant před položením.

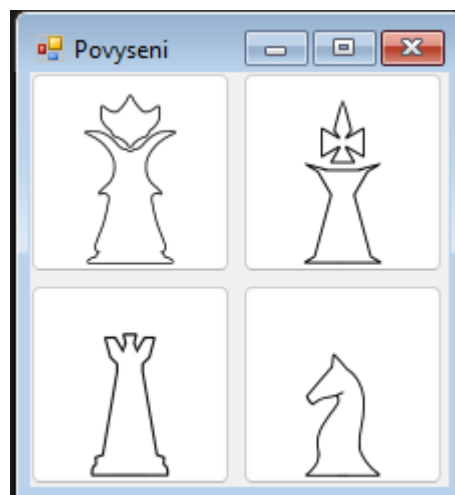
```
public Button[,] enPeasantMove(string kodfigurky, int[] pole, int[] pole2, int[] pole3, Button[,] hraciPole)
{
    hraciPole[pole[0], pole[1]].BackgroundImage = null;
    hraciPole[pole[0], pole[1]].Tag = null;
    hraciPole[pole3[0], pole3[1]].BackgroundImage = null;
    hraciPole[pole3[0], pole3[1]].Tag = null;
    hraciPole[pole2[0], pole2[1]].BackgroundImage = TridaFigurky.rozpoznavaniFigurek(kodfigurky);
    hraciPole[pole2[0], pole2[1]].Tag = kodfigurky;
    return hraciPole;
}
```

Obrázek 32 - položení en passant

Povýšení

Povýšení je tah kdy se pěšák dostane na druhý okraj šachovnice, což mu umožní změnit se na jinou figurku (až na krále nebo pěšáka).

Na vybrání povýšené figurky jsem si vytvořil nové okénko, kde jsou čtyři tlačítka. Na každé tlačítko jsem dal obrázky možných figurek. Když se klikne na některé tlačítko vezme se kód dané figurky a položí se na místo povýšeného pěšáka. Pokud si však nic nevyberete a zavřete okno vybere se automaticky královna. Form se vždy při položení pěšáka otevře, pokud je řádek aktuální souřadnice 0 nebo 7 (okraje šachovnice). Ještě se zkontroluje, pokud je pěšák na správném okraji (bílý pěšák se musí být na řádek 0 a černý a 7 řádek)



Obrázek 33 - okénko povýšení

```
private void PovyseniClick(object sender, EventArgs e)
{
    VybranaFigurka = (sender as Button).Tag.ToString();
    if (hlavniForm.bilyNaRade != true)
    {
        VybranaFigurka = VybranaFigurka.ToLower();
    }

    hlavniForm.stavHracihoPole[hlavniForm.souradniceMinulehoPolicka[0], hlavniForm.souradniceMinulehoPolicka[1]] = 0;
    hlavniForm.stavHracihoPole[hlavniForm.souradniceAktualnihoPolicka[0], hlavniForm.souradniceAktualnihoPolicka[1]] = 1;
    hlavniForm.hraciPole[hlavniForm.souradniceMinulehoPolicka[0], hlavniForm.souradniceMinulehoPolicka[1]].BackgroundImage = null;
    hlavniForm.hraciPole[hlavniForm.souradniceMinulehoPolicka[0], hlavniForm.souradniceMinulehoPolicka[1]].Tag = null;
    hlavniForm.hraciPole[hlavniForm.souradniceAktualnihoPolicka[0], hlavniForm.souradniceAktualnihoPolicka[1]].BackgroundImage = trid
    hlavniForm.hraciPole[hlavniForm.souradniceAktualnihoPolicka[0], hlavniForm.souradniceAktualnihoPolicka[1]].Tag = VybranaFigurka;
    this.Close();
}
}
```

Obrázek 34 - Klik metoda pro tlačítka.

```
private void Povyseni_FormClosed(object sender, FormClosedEventArgs e)
{
    if (VybranaFigurka == null)
    {
        VybranaFigurka = "Q";
        if (hlavniForm.bilyNaRade != true)
        {
            VybranaFigurka = VybranaFigurka.ToLower();
        }

        hlavniForm.stavHracihoPole[hlavniForm.souradniceMinulehoPolicka[0], hlavniForm.souradniceMinulehoPolicka[1]] = 0;
        hlavniForm.stavHracihoPole[hlavniForm.souradniceAktualnihoPolicka[0], hlavniForm.souradniceAktualnihoPolicka[1]] = 1;
        hlavniForm.hraciPole[hlavniForm.souradniceMinulehoPolicka[0], hlavniForm.souradniceMinulehoPolicka[1]].BackgroundImage = null;
        hlavniForm.hraciPole[hlavniForm.souradniceMinulehoPolicka[0], hlavniForm.souradniceMinulehoPolicka[1]].Tag = null;
        hlavniForm.hraciPole[hlavniForm.souradniceAktualnihoPolicka[0], hlavniForm.souradniceAktualnihoPolicka[1]].BackgroundImage = trid
        hlavniForm.hraciPole[hlavniForm.souradniceAktualnihoPolicka[0], hlavniForm.souradniceAktualnihoPolicka[1]].Tag = VybranaFigurka;
    }
    this.Close();
}
```

Obrázek 35 - Vybrání královny v případě, kdy není vybraná figurka.

```
bool spravnaFigurkaNaPovyseni()
{
    return (vybranaFigurka == "P" && souradniceAktualnihoPolicka[0] == 0 && souradniceMinulehoPolicka[0] == souradniceAktualnihoPolicka[0] + 1) ||
           (vybranaFigurka == "p" && souradniceAktualnihoPolicka[0] == 7 && souradniceMinulehoPolicka[0] == souradniceAktualnihoPolicka[0] - 1);
}
```

Obrázek 36 - kontrola správné strany šachovnice pro povýšení.

```
povyseni.ShowDialog();
promoted = true;
Array.Clear(EnPeasant, 0, 2);
```

Obrázek 37 - Otevření povyšovacího okna.

Vymazání tahů, co nechají svého krále v šachu

V šachách nemůžete provést tah, který vašeho krále nechá v šachu.

V aplikaci je třída kde se vždy po vygenerování tahů pro vybranou figurku otestují všechny tahy, jestli nenechají jejich krále v šachu. V oddělené metodě se vezmou vygenerované tahy a otestují se, pokud je podmínka pravda tak se přidá tah do listu, který se pak vrátí pro platné tahy figurky. Udělá se kopie šachovnice, na které se provede tah a zkontroluje se, pokud je král v šachu a vrací pravdu nebo nepravdu.

Kontrola krále v šachu je provedena tak, že se projdou všechny figurky oponenta a podívá se, pokud mají tah na krále. Krále souřadnice se hledají projížděním celé šachovnice a hledání políčka s kódem krále.

Pro en passant a rošádu, jsem musel udělat od dělenou metodu, protože potřebují jiné parametry pro vykonání tahu.

```

public List<int> tahyProtiSachuMatu(int[,] stavPole, Button[,] hraciPole, bool bilyNaRade, Button vybranaFigurka)
{
    List<int> tahyProtiSM = new List<int>();

    if (vybranaFigurka.Tag != null)
    {
        List<int> temp = tahFigurka.vypocetTahu(realizacePohybu.hledacSouradnic(vybranaFigurka, hraciPole), vybran
        for (int i = 0; i < temp.Count; i += 2)
        {
            int[] temp2 = { temp[i], temp[i + 1] };
            if (testTahuProtiSachuMatu(stavPole, hraciPole, bilyNaRade, vybranaFigurka, temp2))
            {
                tahyProtiSM.Add(temp[i]);
                tahyProtiSM.Add(temp[i + 1]);
            }
        }
    }
    return tahyProtiSM;
}

```

Obrázek 38 - zkoušení tahu

Detekce konce hry

Hra může skončit tím, že hráč nemá platné tahy (remíza), hráč nemá platné tahy a jeho král je v šachu (vyhraje jeden z hráčů) nebo hráč nemá dostatek figurek (remíza).

První metoda kontroluje, pokud má hráč nějaké plané tahy. To je provedeno tak, že se projdou všechny figurky a jejich tahy se přidají do listu. Pokud je na konci list prázdný tak se vrací pravda.

Další metoda je při nedostatku materiálu, ten zase projede celou šachovnicí a spočítá figurky jako král, královna, věž atd. Na konci se pak podle předem určených podmínek vrací pravda pokud je nedostatek figurek nebo nepravda při opaku.

V hlavním kódu jsem pak při každém položení figurky zkontroluje, pokud není, již konec hry. Metoda pro počet platných tahů se použije samotná pro remízu nebo společně s metodou pro kontrolu krále v šachu se zase může jednat o výhru jednoho z hráčů. Potom metoda pro nedostatek materiálů je samostatná podmínka.

```

public bool maOponentPlatnePohyby(int[,] stavPole, Button[,] hraciPole, bool bilyNaRade)
{
    List<int> platneTahy = new List<int>();
    Button[,] temphraciPole = new Button[8, 8];
    for (int y = 0; y < hraciPole.GetLength(0); y++)
    {
        for (int x = 0; x < hraciPole.GetLength(1); x++)
        {
            temphraciPole[y, x] = new Button();
            temphraciPole[y, x].Tag = hraciPole[y, x].Tag;
            temphraciPole[y, x].BackgroundImage = hraciPole[y, x].BackgroundImage;
        }
    }
    int[,] tempStavPole = (int[,])stavPole.Clone();

    foreach (Button button in temphraciPole)
    {
        if (button.Tag != null)
        {
            List<int> temp = detekceSachu.tahyProtiSachuMatu(tempStavPole, temphraciPole, bilyNaRade, button);
            foreach (int y in temp)
            {
                platneTahy.Add(y);
            }
        }
    }

    if (platneTahy.Count == 0)
    {
        return true;
    }

    return false;
}

```

Obrázek 39 - vyhodnocení počtu platných tahů


```

public bool nedostatekMaterialu(Button[,] hraciPole)
{
    int WB = 0;
    int BB = 0;
    int WN = 0;
    int BN = 0;
    int bileFigurky = 0;
    int cerneFigurky = 0;

    foreach (Button t in hraciPole)
    {
        if (t.Tag != null)
        {
            string piece = t.Tag.ToString();
            switch (piece)
            {
                case "K":
                    bileFigurky++;
                    break;
                case "k":
                    cerneFigurky++;
                    break;
                case "B":
                    WB++;
                    bileFigurky++;
                    break;
                case "b":
                    BB++;
                    cerneFigurky++;
                    break;
            }
        }
    }
}

```

Obrázek 40 - sčítání materiálu 1/3

```

                case "B":
                    WB++;
                    bileFigurky++;
                    break;
                case "b":
                    BB++;
                    cerneFigurky++;
                    break;
                case "N":
                    WN++;
                    bileFigurky++;
                    break;
                case "n":
                    BN++;
                    cerneFigurky++;
                    break;
            }
            default:
                return false;
        }
    }

    if (bileFigurky == 1 && cerneFigurky == 1)
    {
        return true;
    }
    if (bileFigurky == 2 && cerneFigurky == 1 && (WB == 1 || WN == 1))
    {
        return true;
    }
}

```

Obrázek 41 - sčítání materiálu 2/3

```

    if (bileFigurky == 1 && cerneFigurky == 2 && (BB == 1 || BN == 1))
    {
        return true;
    }
    if (bileFigurky == 2 && cerneFigurky == 2 && WB == 1 && BB == 1)
    {
        return true;
    }
    return false;
}

```

Obrázek 42 - sčítání materiálu 3/3

Výpis tahů do listu

Při každém tahu se vždy zaznamená do listu historie.

Vytvořil jsem si list Box, který se přidá do formu při vytvoření šachovnice a jeho pozice je vázána na velikost šachovnice. Nachází se na pravé straně. Vzor mám podle šachová notace z wikipedie.[\[5\]](#)

Kód pro výpis tahů je v samostatné třídě. K vypíšu vždy dojde při položení figurky. Notace je rozdělena na tři části pro lepší úpravu. Vytvořil jsem si i slovník pro správnou notaci, protože souřadnice, které používám jsou jen číslem. První se v hlavní metodě upraví souřadnice, aby seděli notaci. Poté se podívá, jestli byla sebrána figurka což zjistí sečtením figurek před a po vykonání tahu. Pokud byla sebrána přidá se do notace „x“ jako sebraní figurky poté se přidá upravená souřadnice. Jestli jste provedly rošádu tak se notace vymaže a nahradí se speciální notací „O-O“ nebo „O-O-O“ podle její strany. Pokud jste sebrali figurku přes en passant tak se úplně na konec přidá „e.p.“ nebo při povýšení se na konec přidá „=> {povýšená figurka}“. Nakonec pokud je král oponenta po tahu v šachu přidá se na úplný konec znak „+“.

Poté jsem přidal list, do kterého se ukládá aktuální šachovnice přeložená do textového řetězce. Pro funkci, kdy stisknete na předchozí tah tak vám to ukáže, jak vypadala šachovnice po jejím provedení.

```
Dictionary<int, string[]> spravneSyntax = new Dictionary<int, string[]>()
{
    {0, ["a", "8"]},
    {1, ["b", "7"]},
    {2, ["c", "6"]},
    {3, ["d", "5"]},
    {4, ["e", "4"]},
    {5, ["f", "3"]},
    {6, ["g", "2"]},
    {7, ["h", "1"]},
};
public List<string> historieHracihoPolePriPohybu = new List<string>();
List<int> tempPoziceStejnýchFigurek = new List<int>();
```

Obrázek 43 - slovník správných notací

```
public string zmenaPoleNaString(Button[,] hraciPole)
{
    string poleNaString = "";
    foreach (Button button in hraciPole)
    {
        if (button.Tag != null)
        {
            poleNaString += button.Tag.ToString();
            poleNaString += realizacePohybu.hledacSouradnic(button, hraciPole)[0];
            poleNaString += realizacePohybu.hledacSouradnic(button, hraciPole)[1];
        }
    }
    return poleNaString;
}
```

Obrázek 44 - změna souřadnic na správnou notaci

```

string tahNotace = "";
string zacatek = figurka;
string konec = "";
if (figurka.ToLower() == "p")
{
    if (!bylaFigurkaSebrana(stavPole, stavPolePo))
    {
        zacatek = "";
    }
    if (povyseni)
    {
        string H = bilyNaRade ? povysenafig.Tag.ToString() : povysenafig.Tag.ToString().ToLower();
        konec += $">{H}";
    }
}

if (hledaniPoziceStejnýchTahu(hraciPole, stavPole, bilyNaRade, figurkaBtn, souradnicePole) > 0)
{
    for (int i = 0; i < tempPoziceStejnýchFigurek.Count; i += 2)
    {
        if (souradnicePole[0] != tempPoziceStejnýchFigurek[i] && souradnicePole[1] != tempPoziceStejnýchFigurek[i + 1])
        {
            zacatek += upravaSytaxeTahu(minuleSouradnice)[0] + upravaSytaxeTahu(minuleSouradnice)[1];
            break;
        }

        if (souradnicePole[1] == tempPoziceStejnýchFigurek[i + 1])
        {

```

Obrázek 45 - metoda citace 1/3

```

if (castled)
{
    int[] stranaCasteling = detekceSachu.hledacPoziceKrale(hraciPole, bilyNaRade);
    if (stranaCasteling[1] == 2)
    {
        tahNotace = "0-0-0" + konec;
    }
    else if (stranaCasteling[1] == 6)
    {
        tahNotace = "0-0" + konec;
    }
}

else
{
    tahNotace = zacatek + upravaSytaxeTahu(souradnicePole)[0] + upravaSytaxeTahu(souradnicePole)[1] + konec;
}
historieHracihoPolePriPohybu.Add(zmenaPoleNaString(hraciPole));
return tahNotace;

```

Obrázek 46 - metoda citace 2/3

```

        if (souradnicePole[1] == tempPoziceStejnýchFigurek[i + 1])
        {
            zacatek += upravaSytaxeTahu(minuleSouradnice)[0];
            break;
        }
        if (souradnicePole[0] == tempPoziceStejnýchFigurek[i])
        {
            zacatek += upravaSytaxeTahu(minuleSouradnice)[1];
            break;
        }
    }
}
tempPoziceStejnýchFigurek.Clear();

if (bylaFigurkaSebrana(stavPole, stavPolePo))
{
    if (figurka.ToLower() == "p")
    {
        zacatek = upravaSytaxeTahu(minuleSouradnice)[0];
        if (enPeasant)
        {
            konec += " e.p. ";
        }
    }
    zacatek += "x";
}

if (detekceSachu.jeKralSach(hraciPole, !bilyNaRade, stavPole))
{
    konec = "+";
}

```

Obrázek 47 - metoda citace 3/3

Výpis sebraných figurek

Pro výpis sebraných figurek jsem vytvořil datagridview kam se při sebrání figurek objeví obrázek sebrané figurky. Na detekci sebraných figurek jsem použil tu samou metodu jako předešlé funkci. Figurka se přidá tak, že se udělá nový image kolem ten se upraví na velikost z toho pixelů a dá se do ní obrázek sebrané figurky nakonec se potom přidá do listu černého nebo bílého hráče.

```
private void SebraneFiguryVypis(int[,] tempStavHracihoPole, string sebranaFigurka)
{
    if (vypisTahu.bylaFigurkaSebrana(stavHracihoPole, tempStavHracihoPole))
    {
        if (bilyNaRade)
        {
            DataGridViewImageColumn imageColumn = new DataGridViewImageColumn();
            imageColumn.ImageLayout = DataGridViewImageCellLayout.Zoom;
            imageColumn.Width = 100;
            imageColumn.Image = figurka.rozpoznavaniFigurek(sebranaFigurka);
            cernySebran.Columns.Add(imageColumn);
            cernyCapturedString.Add(sebranaFigurka);
        }
        else
        {
            DataGridViewImageColumn imageColumn = new DataGridViewImageColumn();
            imageColumn.ImageLayout = DataGridViewImageCellLayout.Zoom;
            imageColumn.Width = 100;
            imageColumn.Image = figurka.rozpoznavaniFigurek(sebranaFigurka);
            bilySebran.Columns.Add(imageColumn);
            bilyCapturedString.Add(sebranaFigurka);
        }
    }
}
```

Obrázek 48 - výpis sebraných figurek

Metoda nová hra

Pokud hra skončí tak jsem přidal metodu, kde všechny proměnné a šachovnice vyresetují pro novou hru.

```
private void novaHra()
{
    cislotahu = 0;
    ChessKomp.endGameReset();
    cernySebran.Columns.Clear();
    bilySebran.Columns.Clear();
    poleTahy.Items.Clear();
    vypisTahu.vymazPriKonci();
    cernyCapturedString.Clear();
    bilyCapturedString.Clear();
    KingBMove = false;
    KingWMove = false;
    Array.Clear(EnPeasant, 0, 2);
    Array.Clear(RookMoves, 0, 4);
    kresleniFigurek(PostaveniFigurek, hraciPole);
    bilyNaRade = true;
    resetovaniBarevPolicek();
    enpeasantChecker = false;
    castled = false;
    vybranaFigurka = null;
    promoted = false;
    resetCaptueGrid();
}
```

Obrázek 49 - reset hry na novou

Počítačový oponent

První verze

Začal jsem pracovat na počítačovém oponentu, který hraje proti vám. Je implementován tak, že se vezme náhodná figurka a její platné tahy. Tahy se poté seřadí podle důležitosti. Seřazeny jsou následovně záchytné tahy, tahy, kde je král oponenta v šachu a ostatní tahy. Pokud jsou záchytné tahy prázdné přejde se na další tahy. Pokud se vybere tah, kdy se pěšák dostane na druhou stranu šachovnice, automaticky se povýší na královnu.

Na konci se potom vrací kód vybrané figurky a souřadnice pro další zpracování (výpis tahů nebo sebraná figurka).

```
public Button[,] AI(Button[,] hraciPole, int[,] stavHracihoPole, bool bilyNaRade)
{
    povyseni = false;
    Random rand = new Random();
    bool vybrani = true;
    List<Button> platneFigurky = new List<Button>();
    foreach (var item in hraciPole)
    {
        if (item.Tag != null)
        {
            if ((Char.IsLower(item.Tag.ToString()[0]) && !bilyNaRade) || (!Char.IsLower(item.Tag.ToString()[0]) && bilyNaRade))
            {
                platneFigurky.Add(item);
            }
        }
    }
}
```

Obrázek 50 - náhodný počítač 1/4

```
while (vybirani && platneFigurky.Count > 0)
{
    Button vybranaFigurka = platneFigurky[rand.Next(platneFigurky.Count)];
    List<int> platneTahy = detekceSachu.tahyProtiSachuMatu(stavHracihoPole, hraciPole, bilyNaRade, vybranaFigurka);

    if (platneTahy.Count > 1)
    {
        List<int[]> captureMoves = new List<int[]>();
        List<int[]> sachMoves = new List<int[]>();
        List<int[]> otherMoves = new List<int[]>();

        for (int i = 0; i < platneTahy.Count; i += 2)
        {
            int[] aktualniSouradniceTemp = { platneTahy[i], platneTahy[i + 1] };
            if (hraciPole[aktualniSouradniceTemp[0], aktualniSouradniceTemp[1]].BackgroundImage != null)
            {
                captureMoves.Add(aktualniSouradniceTemp);
            }
            else if (detekceSachu.testTahuProtiSachuMatu(stavHracihoPole, hraciPole, !bilyNaRade, vybranaFigurka, aktualniSouradniceTemp))
            {
                sachMoves.Add(aktualniSouradniceTemp);
            }
            else
            {
                otherMoves.Add(aktualniSouradniceTemp);
            }
        }
    }
}
```

Obrázek 51 - náhodný počítač 2/4

```
int[] selectedMove;
if (captureMoves.Count > 0)
{
    selectedMove = captureMoves[rand.Next(captureMoves.Count)];
}
else if (sachMoves.Count > 0)
{
    selectedMove = sachMoves[rand.Next(captureMoves.Count)];
}
else
{
    selectedMove = otherMoves[rand.Next(otherMoves.Count)];
}

int[] minuleSouradnice = realizacePohybu.hledacSouradnic(vybranaFigurka, hraciPole);
int[] aktualniSouradnice = selectedMove;
```

Obrázek 52 - náhodný počítač 3/4

```
hraciPole = realizacePohybu.polozeniFigurek(vybranaFigurka.Tag.ToString(), minuleSouradnice, aktualniSouradnice, hraciPole);
```

Obrázek 53 - náhodný počítač 4/4

Druhá verze

Ve druhé verzi jsem přidal evaluační třídu, která vyhodnotí skóre aktuální pozice šachovnice. Evaluace funguje tak, že se sečtou všechny typy a vynásobí jednotlivé druhy jejich daným skórem např. pěšák je 250 bodů. Nakonec se přičte skóre jednotlivých pozic figurek, což znamená, že se pěšák nerovná 250 bodů, ale 255, protože je uprostřed šachovnice. To samé se udělá i s oponentovými figurkami a odečtou se jejich skóre, aby vyšlo celkové skóre pozice.

Figurku jsi pořád vybere náhodně, její tahy provede na kopii šachovnice a vyhodnotí její pozici. Tah, kde je pozice s nevyšším skóre se vybere a zahraje na šachovnici. [\[6\]](#)

```
public const int PawnValue = 100;
public const int KnightValue = 300;
public const int BishopValue = 320;
public const int RookValue = 500;
public const int QueenValue = 900;

private static readonly int[,] pawn =
{
    { 0, 0, 0, 0, 0, 0, 0, 0, },
    { 50, 50, 50, 50, 50, 50, 50, 50, },
    { 10, 10, 20, 30, 30, 20, 10, 10, },
    { 5, 5, 10, 25, 25, 10, 5, 5, },
    { 0, 0, 0, 20, 20, 0, 0, 0, },
    { 5, -5, -10, 15, 15, -10, -5, 5, },
    { 5, 10, 10, -10, -10, 10, 10, 5, },
    { 0, 0, 0, 0, 0, 0, 0, 0, },
};
```

Obrázek 54 - hodnota figurek a příklad bonusu souřadnic pěšáka

```

int[,] flippedPawn = druhaStrana(pawn);
int[,] flippedPawnsEnd = druhaStrana(PawnsEnd);
int[,] flippedRooks = druhaStrana(Rooks);
int[,] flippedKnights = druhaStrana(Knights);
int[,] flippedBishops = druhaStrana(Bishops);
int[,] flippedQueens = druhaStrana(Queens);

6 references
static int[,] druhaStrana(int[,] table)
{
    int[,] flippedTable = new int[8, 8];

    for (int y = 0; y < 8; y++)
    {
        for (int x = 0; x < 8; x++)
        {
            flippedTable[y, x] = table[7 - y, 7 - x];
        }
    }

    return flippedTable;
}

```

Obrázek 55 - prohození tabulek pro černého

```

switch (figurka.ToLower())
{
    case "p":
        poziceValue = endgame ? (bilyNaRade ? PawnsEnd[pozice[0], pozice[1]] : flippedPawnsEnd[pozice[0],
        figurkaValue = PawnValue;

        int drahaPromote = bilyNaRade ? pozice[0] : 7 - pozice[0];
        poziceValue += bilyNaRade ? (7 - drahaPromote) * 5 : -((7 - drahaPromote) * 5);

        if ((bilyNaRade && pozice[0] == 0) || (!bilyNaRade && pozice[0] == 7))
        {
            figurkaValue = QueenValue;
        }
        break;
    case "r":
        poziceValue = bilyNaRade ? Rooks[pozice[0], pozice[1]] : flippedRooks[pozice[0], pozice[1]];
        figurkaValue = RookValue;
        break;
    case "n":
        poziceValue = bilyNaRade ? Knights[pozice[0], pozice[1]] : flippedKnights[pozice[0], pozice[1]];
        figurkaValue = KnightValue;
        break;
    case "b":
        poziceValue = bilyNaRade ? Bishops[pozice[0], pozice[1]] : flippedBishops[pozice[0], pozice[1]];
        figurkaValue = BishopValue;
        break;
    case "q":
        poziceValue = bilyNaRade ? Queens[pozice[0], pozice[1]] : flippedQueens[pozice[0], pozice[1]];
        figurkaValue = QueenValue;
        break;
}

```

Obrázek 56 - metoda evaluace 1/2

```

case "k":
    poziceValue = endgame ? KingEnd[pozice[0], pozice[1]] : KingStart[pozice[0], pozice[1]];
    if (Char.IsUpper(figurka[0]))
    {
        poziceWKral = pozice;
    }
    else
    {
        poziceBKral = pozice;
    }
    break;
}

if (Char.IsUpper(figurka[0]))
{
    skore += figurkaValue + poziceValue;
}
else
{
    skore -= figurkaValue + poziceValue;
}

```

Obrázek 57 - metoda evaluace 2/2

Třetí verze

Ve třetí verzi jsem upravil počítač, aby vzal každý tah zahrál na kopii šachovnice a vyhodnotil skóre za pomocí algoritmu MinMax a poté vrátí tah. Poté vybere tah s nejlepším skórem a zahraje ho na opravdové šachovnici. [\[7\]](#)

```
for (int depth = 1; depth <= maxDepth; depth++)
{
    platneTahy = srovnatTahy(board, stavHracihoPole, bilyNaRade);
    foreach (TAH move in platneTahy)
    {
        int[] from = move.From;
        int[] To = move.To;
        var originalBg = board[To[0], To[1]].BackgroundImage;
        var originalTag = board[To[0], To[1]].Tag;
        var figurkaTag = board[from[0], from[1]].Tag.ToString();
        polozeniFigurek(figurkaTag, from, To, board);
        int[,] tempStavPole = updateStavHracihoPole(board);

        double score = Minimax(board, tempStavPole, depth, !bilyNaRade, double.NegativeInfinity, double.PositiveInfinity);

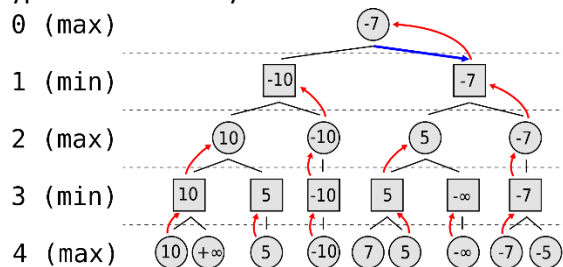
        polozeniFigurek(figurkaTag, To, from, board);
        board[To[0], To[1]].BackgroundImage = originalBg;
        board[To[0], To[1]].Tag = originalTag;

        if ((bilyNaRade && score > bestScore) || (!bilyNaRade && score < bestScore))
        {
            bestScore = score;
            MoveTo = To;
            vybranaFigurka = board[from[0], from[1]];
        }
        if (timelimit.ElapsedMilliseconds > 10000)
        {
            break;
        }
    }
}
```

Obrázek 58 - procházení všech dostupných tahu a hodnocení

MinMax

Minimax je algoritmus používaný pro hraní strategických her mezi dvěma a více hráči. Slouží k nalezení nejlepšího možného tahu v dané pozici. Funguje tak, že postupně simuluje všechny možné tahy a po každém tahu předpokládá, že protivník zahraje svůj nejlepší možný tah. Tento proces se opakuje, dokud se nedosáhne určité hloubky nebo konečného stavu hry. Poté se vyhodnotí skóre pozice z pohledu protivníka – čím lepší je jeho skóre, tím horší je tah pro nás. Na základě těchto výsledků se zpětně vybere nejlepší tah pro počítač. Algoritmus není náročný na paměť, ale může být výpočetně náročný kvůli velkému množství možných tahů, které je třeba analyzovat. [\[7\]](#)



Obrázek 59 - vizualizace minmax algoritmu

```

2:references
private double Minimax(Button[,] board, int[,] stavHracihoPole, int depth, bool bilyNaRade, double alpha, double beta)
{
    string boardHash = GetBoardHash(board, bilyNaRade);
    if (transpositionTable.ContainsKey(boardHash))
    {
        return transpositionTable[boardHash];
    }

    if (depth == 0)
    {
        double evaluation = captureMinMax(board, stavHracihoPole, bilyNaRade, alpha, beta);
        transpositionTable[boardHash] = evaluation;
        return evaluation;
    }

    platneTahy = srovnatTahy(board, stavHracihoPole, bilyNaRade);
    if (platneTahy.Count <= 0) return bilyNaRade ? double.NegativeInfinity : double.PositiveInfinity;
    if (jeKralSach(board, bilyNaRade, platneTahy))
    {
        depth++;
    }
}

```

Obrázek 60 - minmax algoritmus 1/2

```

foreach (TAH move in platneTahy)
{
    int[] From = move.From;
    int[] To = move.To;
    var originalBg = board[To[0], To[1]].BackgroundImage;
    var originalTag = board[To[0], To[1]].Tag;
    var playedPieceTag = board[From[0], From[1]].Tag.ToString();

    polozeniFigurek(playedPieceTag, From, To, board);
    int[,] tempStavPole = updateStavHracihoPole(board);

    double score = Minimax(board, tempStavPole, depth - 1, !bilyNaRade, -beta, -alpha);

    polozeniFigurek(playedPieceTag, To, From, board);
    board[To[0], To[1]].BackgroundImage = originalBg;
    board[To[0], To[1]].Tag = originalTag;

    if (score >= beta)
    {
        return beta;
    }

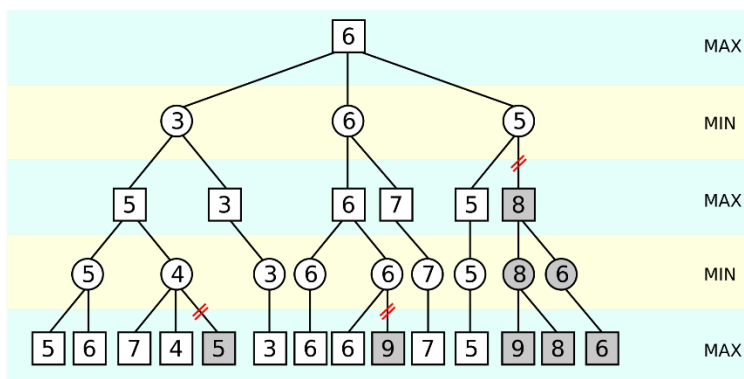
    alpha = Math.Max(alpha, score);
}
transpositionTable[boardHash] = alpha;
return alpha;
}

```

Obrázek 61 - minmax algoritmus 2/2

Přidal jsem do algoritmu MinMax alfa-beta ořezávání, což by mělo zkrátit čas vyhodnocování obětování teoretických lepších výsledků.

Princip alfa-beta ořezávání spočívá v tom, že během simulace tahů sledujeme dvě hodnoty – alfa a beta. Alfa představuje nejlepší známé skóre pro max hráče (tedy počítač), zatímco beta reprezentuje nejlepší známé skóre pro min hráče (tedy protivníka). Jakmile se při prozkoumávání některé větve stromu zjistí, že je zaručené, že daná větev nebude mít lepší výsledek než už prozkoumané tahy, algoritmus tuto větev ořízne a nepokračuje v její analýze. Tím se šetří výpočetní čas, protože se vyhodnotí pouze relevantní tahy. [8]



Obrázek 62 - vizualizace minmax s alfa-beta ořezáváním

```
if (score >= beta)
{
    return beta;
}

alpha = Math.Max(alpha, score);
```

Obrázek 63 - alfa-beta ořezávání v kódu

Transpoziční stoly

Je velmi jednoduché vylepšení Alfa-Beta ořezávání. Funguje to tak, že při každém vyhodnocení pozice se uloží. Pokud MinMax narazí na pozici uloženou v transpozičního stolu, tak nemusíme vyhodnocovat znova tuto pozici a můžeme ušetřit čas tím, že jenom vrátíme její uložené skóre. [\[9\]](#)

```
string boardHash = GetBoardHash(board, bilyNaRade);
if (transpositionTable.ContainsKey(boardHash))
{
    return transpositionTable[boardHash];
}

if (depth == 0)
{
    double evaluation = captureMinMax(board, stavHracihoPole, bilyNaRade, alpha, beta);
    transpositionTable[boardHash] = evaluation;
    return evaluation;
}
```

Obrázek 64 - uložení nové nebo vrácení existující pozice

```
private string GetBoardHash(Button[,] board, bool bilyNaRade)
{
    var sb = new System.Text.StringBuilder(65);

    for (int y = 0; y < 8; y++)
    {
        for (int x = 0; x < 8; x++)
        {
            sb.Append(board[y, x].Tag?.ToString() ?? " ");
        }
    }

    sb.Append(bilyNaRade ? 'W' : 'B');
    return sb.ToString();
}
```

Obrázek 65 - Generování unikátního hash pro pozici

Seřazení tahů podle důležitosti

Seřazení tahů slouží ke zvýšení šance střihu v Alfa-Beta ořezávání tím, že lepší tahy jako sebrání figurky se zařadí na první místa v listu.

```
private List<TAH> srovnatTahy(Button[,] hraciPole, int[,] stavPole, bool bilyNaRade)
{
    List<TAH> tahy = GeneraceTahuAI(stavPole, hraciPole, bilyNaRade, false);
    if (tahy.Count <= 0) { return tahy; }

    sebraniList = new List<TAH>();
    sach = new List<TAH>();
    povyseniList = new List<TAH>();
    ostatni = new List<TAH>();

    foreach (TAH move in tahy)
    {
        int[] souradnice = move.To;
        string vybranaFigurka = hraciPole[move.From[0], move.From[1]].Tag.ToString();
        string captureFigurka = hraciPole[souradnice[0], souradnice[1]].Tag?.ToString();

        if (detekceSachu.testTahuProtiSachuMatu(stavPole, hraciPole, !bilyNaRade, hraciPole[move.From[0], move.From[1]], souradnice))
        {
            sach.Add(move);
        }
        else if (captureFigurka != null && ((Char.IsUpper(captureFigurka[0]) && !bilyNaRade) || (Char.IsLower(captureFigurka[0]) && bilyNaRade)))
        {
            sebraniList.Add(move);
        }
        else if ((souradnice[0] == 0 || souradnice[0] == 7) && vybranaFigurka.ToLower() == "p")
        {
            povyseniList.Add(move);
        }
        else
        {
            ostatni.Add(move);
        }
    }

    tahy.Clear();
    tahy.AddRange(sebraniList);
    tahy.AddRange(sach);
    tahy.AddRange(povyseniList);
    tahy.AddRange(ostatni);

    return tahy;
}
```

Obrázek 66 - srovnání tahů

Quiescence hledání

Předtím MinMax na konci vždy vracel evaluaci šachovnice, ale to nemusí vracet správné hodnocení tak jsem přidal quiescence hledání. Quiescence hledání je jenom složitější název pro algoritmus, který funguje stejně jako minmax jenom prochází jen tahy kde lze sebrat figurku. Tento algoritmus však nepotřebuje danou hloubku prochází tahy, dokud nejsou další. Tento algoritmus nám pomáhá s přesnějším vyhodnocením stavu šachovnice. [\[10\]](#)

```
private double captureMinMax(Button[,] board, int[,] stavHracihoPole, bool bilyNaRade, double alpha, double beta)
{
    double standPat = evaluateFigurek.evaluate(board, bilyNaRade, stavHracihoPole);
    double Bestscore = standPat;
    if (standPat >= beta)
    {
        return standPat;
    }
    alpha = Math.Max(alpha, standPat);

    List<TAH> tahy = GeneraceTahuAI(stavHracihoPole, board, bilyNaRade, true);
    if (tahy.Count <= 0) { return alpha; }
    foreach (TAH move in tahy)
    {
        int[] From = move.From;
        int[] To = move.To;
        var originalBg = board[To[0], To[1]].BackgroundImage;
        var originalTag = board[To[0], To[1]].Tag;
        var playedPieceTag = board[From[0], From[1]].Tag.ToString();

        polozeniFigurek(playedPieceTag, From, To, board);

        double score = captureMinMax(board, stavHracihoPole, !bilyNaRade, -beta, -alpha);

        polozeniFigurek(playedPieceTag, To, From, board);
        board[To[0], To[1]].BackgroundImage = originalBg;
        board[To[0], To[1]].Tag = originalTag;

        if (score >= beta)
        {
            return score;
        }
        Bestscore = Math.Max(score, Bestscore);
        alpha = Math.Max(score, alpha);
    }
    return Bestscore;
}
```

Obrázek 67 - Quiescence hledání

Upravení evaluace šachovnice

Evaluaci jsem upravil, tak aby ke konci hry byl král nucen být blízko nepřátelského krále pro šachmat. A přidal bonusové body za více platných tahů.

```
if (endgame && poziceWKral != null && poziceBKral != null)
{
    int drahaKral = Math.Abs(poziceWKral[0] - poziceBKral[0]) + Math.Abs(poziceWKral[1] - poziceBKral[1]);
    skore += bilyNaRade ? drahaKral * 2 : -(drahaKral * 2);
}
```

Obrázek 68 - bonusové body za menší vzdálenost mezi králi

```
private static bool isEndgame(Button[,] hraciPole)
{
    int queenCount = 0, minorPieceCount = 0;
    foreach (Button b in hraciPole)
    {
        if (b.Tag == null) continue;
        string figurka = b.Tag.ToString().ToLower();
        if (figurka == "q") queenCount++;
        if (figurka == "r" || figurka == "n" || figurka == "b") minorPieceCount++;
    }
    return queenCount <= 1 && minorPieceCount <= 3;
}
```

Obrázek 69 - detekce end game

Předefinované tahy – čtvrtá verze

Jako poslední úpravu jsem přidal menší list odehraných her ze stránky „The Week in Chess“. Počítač si prvních pět tahů vybere z listu, pokud se najde hra, která obsahuje všechny minulé tahy. List je přeložen z PGN (portable game notation) do prvních 10 tahů. Každý řádek je jedna hra. [\[11\]](#)

Při startu hry se načte ze souboru list her a uloží se do slovníku. Když je počítač na řadě projde všechny hry ve slovníku a projde, pokud obsahují všechny minulé tahy je důležité, že jsou tahy i ve stejném pořadí. Pokud se najde hra, která tyto podmínky splňuje uloží se do listu vybraných her. Potom se vybere náhodně jedna hra z vybraných her a vrátí její klíč. V metodě počítačového hráče je podmínka, pokud se najde klíč hry tak se projdou všechny figurky, které se posunout na další tah v listu. Vybere se potom nejlépe hodnocený tah a zahraje ho. Pokud se však nenajde předdefinovaná hra tak se přejde na minmax algoritmus.

```
private void LoadOpenings(string soubor)
{
    string[] lines = File.ReadAllLines(soubor);
    int Index = 0;

    foreach (string line in lines)
    {
        string[] moves = line.Split(' ');
        List<int[]> moveList = new List<int[]>();

        foreach (string move in moves)
        {
            if (move.Length == 2)
            {
                int from = move[0] - '0';
                int to = move[1] - '0';
                moveList.Add([ from, to ]);
            }
        }

        NacteneHry[Index++] = moveList;
    }
}
```

Obrázek 70 - načtení her

```
private int zjistiHru(List<int[]> posledniTah)
{
    if (AktualniTahIndex >= 10) { return -1; }
    Random random = new Random();
    List<int> matchingIndices = new List<int>();

    foreach (var kvp in NacteneHry)
    {
        int index = kvp.Key;
        List<int[]> game = kvp.Value;
        if (game.Count <= AktualniTahIndex) continue;

        bool match = true;
        for (int j = 0; j < posledniTah.Count; j++)
        {
            if (!posledniTah[j].SequenceEqual(game[j]))
            {
                match = false;
                break;
            }
        }

        if (match)
        {
            matchingIndices.Add(index);
        }
    }

    if (matchingIndices.Count > 0)
    {
        return matchingIndices[random.Next(matchingIndices.Count)];
    }

    return -1;
}
```

Obrázek 71 - zjištění stejných her

```

int klichHry = zjistitHru(posledniTahyList);
if (klichHry != -1)
{
    platneTahy = srovnatTahy(board, stavHracihoPole, bilyNaRade);
    int[] tempMove = NacteneHry[klichHry][AktualnitTahIndex];
    foreach (TAH move in platneTahy)
    {
        if (move.To[0] == tempMove[0] && move.To[1] == tempMove[1])
        {
            int[] from = move.From;
            int[] To = move.To;
            var originalBg = board[To[0], To[1]].BackgroundImage;
            var originalTag = board[To[0], To[1]].Tag;
            var figurkaTag = board[from[0], from[1]].Tag.ToString();
            polozeniFigurek(figurkaTag, from, To, board);
            int[,] tempStavPole = updateStavHracihoPole(board);

            double score = evaluateFigurek.evaluate(board, bilyNaRade, tempStavPole);

            polozeniFigurek(figurkaTag, To, from, board);
            board[To[0], To[1]].BackgroundImage = originalBg;
            board[To[0], To[1]].Tag = originalTag;
            if ((bilyNaRade && score > bestScore) || (!bilyNaRade && score < bestScore))
            {
                bestScore = score;
                MoveTo = To;
                vybranaFigurka = board[from[0], from[1]];
            }
        }
    }
}

```

Obrázek 72 - odehrání dalšího tahu

Uložení/načtení her

Jako poslední funkci, na které jsem pracoval bylo načten nebo uložení hry.

Hra a její důležité parametry jako en pasant, rošáda nebo kdo je neřadě se uloží do XML souboru. Jednotlivé parametry jsou rozděleny do jednotlivých kategorií pro lepší orientaci při načítání. Kam si chcete uložit soubor se dá vybrat pomocí průzkumníka souborů, který se při uložení spustí. To samé se stane i při načítání hry. Informace ze XML souboru se čtou podle jednotlivých kategorií.

```

private void LoadXml(object sender, EventArgs e)
{
    OpenFileDialog openFileDialog = new OpenFileDialog
    {
        Filter = "XML Files|*.xml",
        Title = "Načti save"
    };
    if (openFileDialog.ShowDialog() == DialogResult.OK)
    {
        try
        {
            XmlDocument xmlDocument = XmlDocument.Load(openFileDialog.FileName);

            novaHra();
            kresleniFigurek(xmlDocument.Root.Name.LocalName, hraciPole);
            var pozice = xmlDocument.Descendants("Pozice").Elements("Item").Select(item => item.Value).ToList();
            vypisTahu.sebratListPozic(pozice);

            var tahy = xmlDocument.Descendants("Tahy").Elements("Item").Select(item => item.Value).ToList();
            foreach (var v in tahy)
            {
                poleTahy.Items.Add(v);
            }

            var bilyNaRade1 = xmlDocument.Descendants("bilyNaRade").FirstOrDefault()?.Value;
            var kingBMove1 = xmlDocument.Descendants("KingBMove").FirstOrDefault()?.Value;
            var kingWMove1 = xmlDocument.Descendants("KingWMove").FirstOrDefault()?.Value;
            bilyNaRade = Convert.ToBoolean(bilyNaRade1);
            KingBMove = Convert.ToBoolean(kingBMove1);
            KingWMove = Convert.ToBoolean(kingWMove1);
        }
        catch { }
    }
}

```

Obrázek 73 - načtení hry 1/3

```

var rookMoves1 = xmlDocument.Descendants("RookMoves").Elements("Item").Select(item => item.Value).ToList();
for (int i = 0; i < rookMoves1.Count; i++)
{
    if (rookMoves1[i] != "N")
    {
        RookMoves[i] = null;
    }
    else
    {
        RookMoves[i] = "N";
    }
}

var cernyCapturedString1 = xmlDocument.Descendants("cernyCapturedString").Elements("Item").Select(item => item.Value).ToList();
foreach (var c in cernyCapturedString1)
{
    DataGridViewImageColumn imageColumn = new DataGridViewImageColumn();
    imageColumn.ImageLayout = DataGridViewImageCellLayout.Zoom;
    imageColumn.Width = 100;
    imageColumn.Image = figurka.rozpoznavaniFigurek(c);
    cernySebran.Columns.Add(imageColumn);
    cernyCapturedString.Add(c);
}

var bilyCapturedString1 = xmlDocument.Descendants("bilyCapturedString").Elements("Item").Select(item => item.Value).ToList();
foreach (var c in bilyCapturedString1)
{
    DataGridViewImageColumn imageColumn = new DataGridViewImageColumn();
    imageColumn.ImageLayout = DataGridViewImageCellLayout.Zoom;
    imageColumn.Width = 100;
    imageColumn.Image = figurka.rozpoznavaniFigurek(c);
    bilySebran.Columns.Add(imageColumn);
    bilyCapturedString.Add(c);
}

```

Obrázek 74 - načtení hry 2/3

```

var Enpeasant1 = xmlDocument.Descendants("Enpeasant").Elements("Item").Select(item => item.Value).ToList();
EnPeasant[0] = Convert.ToInt32(Enpeasant1[0]);
EnPeasant[1] = Convert.ToInt32(Enpeasant1[1]);

MessageBox.Show("\n" + openFileDialog.FileName, "Soubor se úspěšně načetl");
}
catch (Exception ex)
{
    MessageBox.Show("Chyba při načtení: " + ex.Message);
}

```

Obrázek 75 - načtení hry 3/3

```

private void SaveXml(object sender, EventArgs e)
{
    SaveFileDialog saveFileDialog = new SaveFileDialog
    {
        Filter = "XML Files|*.xml",
        Title = "Ulož si hru"
    };

    if (saveFileDialog.ShowDialog() == DialogResult.OK)
    {
        try
        {
            List<string> tahy = new List<string>();
            foreach (var i in poleTahy.Items)
            {
                tahy.Add(i.ToString());
            }

            List<string> pozice = vypisTahu.vratitListPozici();
            List<string> RookMovesTemp = new List<string>();
            foreach (string i in RookMoves)
            {
                if (i != "N")
                {
                    RookMovesTemp.Add("A");
                }
                else
                {
                    RookMovesTemp.Add(i);
                }
            }
        }
    }
}

```

Obrázek 76 - uložení hry 1/2

```

XDocument xmlDocument = new XDocument(
    new XElement("Pozice", pozice.Count - 1),
    new XElement("Pozice", pozice.Select(p => new XElement("Item", p))),
    new XElement("Tahy", tahy.Select(t => new XElement("Item", t))),
    new XElement("bilyNaRade", [bilyNaRade]),
    new XElement("KingBMove", [KingBMove]),
    new XElement("KingWMove", [KingWMove]),
    new XElement("RookMoves", RookMovesTemp.Select(t => new XElement("Item", t))),
    new XElement("cernyCapturedString", cernyCapturedString.Select(t => new XElement("Item", t))),
    new XElement("bilyCapturedString", bilyCapturedString.Select(t => new XElement("Item", t))),
    new XElement("EnPeasant", EnPeasant.Select(t => new XElement("Item", t)))
);
xmlDocument.Save(saveFileDialog.FileName);
MessageBox.Show("Uloženo");
}
catch (Exception ex)
{
    MessageBox.Show("Error při uložení: " + ex.Message, "Error");
}

```

Obrázek 77 - uložení hry 2/2

Závěr

Během vývoje bylo nutné řešit řadu výzev, od implementace základních pravidel šachu až po optimalizaci výpočetního výkonu při simulaci tahů počítačového oponenta. Klíčovým prvkem byla implementace algoritmů MinMax a Alfa-Beta ořezávání, které umožňují počítači hrát na slušné úrovni. Tyto algoritmy byly vylepšeny pomocí transpozičních tabulek a quiescence search, což zvýšilo přesnost výpočtů. Projekt přinesl cenné zkušenosti v oblasti objektově orientovaného programování, práce s grafikou a optimalizace kódu, a zároveň poskytl hlubší pochopení šachových pravidel a logiky, která stojí za šachovými stroji.

Aplikace má potenciál pro další rozšíření, jako je hra přes síť, která by umožnila hráčům hrát proti sobě online, nebo implementace složitějších AI oponentů. Dalším možným vylepšením by byla analýza her, která by hráčům pomohla zlepšit jejich strategii a pochopit chyby. Přidání podpory pro různé šachové varianty, jako je šach 960 nebo rychlý šach, by také rozšířilo možnosti hry.

Projekt byl úspěšně dokončen a splnil všechny stanovené cíle. Vývoj této aplikace přinesl cenné zkušenosti a otevřel dveře pro další vylepšení a rozšíření funkcionality.

Citace

1. MICROSOFT. *C Sharp*. Online. Wikipedie. 2024. Dostupné z: https://cs.wikipedia.org/wiki/C_Sharp. [cit. 2025-03-08].
 2. MICROSOFT. *Microsoft Visual Studio*. Online. Wikipedie. 2025. Dostupné z: https://cs.wikipedia.org/wiki/Microsoft_Visual_Studio. [cit. 2025-03-08].
 3. SAVAGE INTERACTIVE. *Procreate (software)*. Online. Wikipedie. 2025. Dostupné z: [https://en.wikipedia.org/wiki/Procreate_\(software\)](https://en.wikipedia.org/wiki/Procreate_(software)). [cit. 2025-03-08].
 4. CHESS.COM, LLC. *Chess*. Online. Chess.com. 2007. Dostupné z: <https://www.chess.com/>. [cit. 2025-03-08].
 5. WIKIPEDIA. *Šachová notace*. Online. 2024. Dostupné z: [https://en.wikipedia.org/wiki/Algebraic_notation_\(chess\)](https://en.wikipedia.org/wiki/Algebraic_notation_(chess)). [cit. 2025-03-15].
 6. CHESSPROGRAMMING WIKI. *Evaluce pozice*. Online. 2024. Dostupné z: <https://www.chessprogramming.org/Evaluation>. [cit. 2025-03-15].
 7. WIKIPEDIA. *MinMax algoritmus*. Online. 2023. Dostupné z: [https://cs.wikipedia.org/wiki/Minimax_\(algoritmus\)](https://cs.wikipedia.org/wiki/Minimax_(algoritmus)). [cit. 2025-03-15].
- CHESSPROGRAMMING WIKI. *Negamax*. Online. 2024. Dostupné z: <https://www.chessprogramming.org/Negamax>. [cit. 2025-03-15].
8. WIKIPEDIA. *Alfa-beta ořezávání*. Online. 2023. Dostupné z: <https://cs.wikipedia.org/wiki/Alfa-beta>. [cit. 2025-03-15].
 9. CHESSPROGRAMMING WIKI. *Transpoziční stoly*. Online. 2024. Dostupné z: https://www.chessprogramming.org/Transposition_Table#How_it_works. [cit. 2025-03-15].
 10. WIKIPEDIE. *Quiescence search*. Online. 2024. Dostupné z: https://en.wikipedia.org/wiki/Quiescence_search. [cit. 2025-03-15].
 11. THE WEEK IN CHESS. *TWIC Archive*. Online. 1994, 2025-03-10. Dostupné z: <https://theweekinchess.com/twic>. [cit. 2025-03-15].
 12. CITACE PRO. *Citace*. Online. 2015, 2025. Dostupné z: <https://www.citacepro.com>. [cit. 2025-03-15].
 13. DRAWIO. *Diagramy*. Online. ---. Dostupné z: <https://www.drawio.com/>. [cit. 2025-03-15].
 14. LUCIDCHART. *UML Activity Diagram*. Online. 2025. Dostupné z: <https://www.lucidchart.com/pages/uml-activity-diagram>. [cit. 2025-03-15].
 15. LUCIDCHART. *UML Use Case Diagram*. Online. 2025. Dostupné z: <https://www.lucidchart.com/pages/uml-use-case-diagram>. [cit. 2025-03-15].